

Instituto Politécnico Nacional

*Unidad Profesional Interdisciplinaria en Ingeniería y
Tecnologías Avanzadas*

Ingeniería Telemática
Bases de Datos Distribuidas
Prof. Carlos de la Sosa Cruz
Grupo 3TM2

Practica No. 4
Problemas de concurrencia

Integrantes:

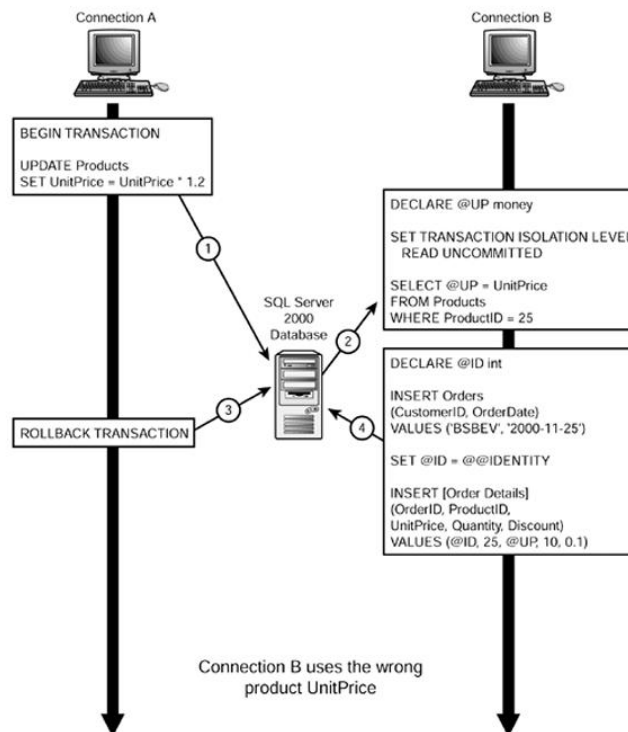
Nieves Hernandez Efrain
Rodríguez Escalera Ricardo
Sánchez Lemus Ulises Mariano

Lectura sucia (Dirty read).

Ocurre cuando un proceso o transacción (Conexión A) lee datos que han sido modificados por otra transacción (Conexión B) que aún no ha realizado un COMMIT. Si la Conexión B decide cancelar sus cambios con un ROLLBACK, la Conexión A habrá trabajado con datos provisionales no válidos.

Sucede cuando una consulta se ejecuta bajo el nivel de aislamiento READ UNCOMMITTED o utiliza la sugerencia/hint de bloqueo NOLOCK / READUNCOMMITTED. Bajo esta condición, SQL Server lee los datos sin solicitar ni respetar los bloqueos exclusivos held por otras transacciones.

SQL Server evita este problema de forma predeterminada mediante el nivel de aislamiento READ COMMITTED. Este nivel aplica bloqueos exclusivos sobre los datos modificados, obligando a las lecturas concurrentes a esperar hasta que la transacción confirme o descarte sus cambios.

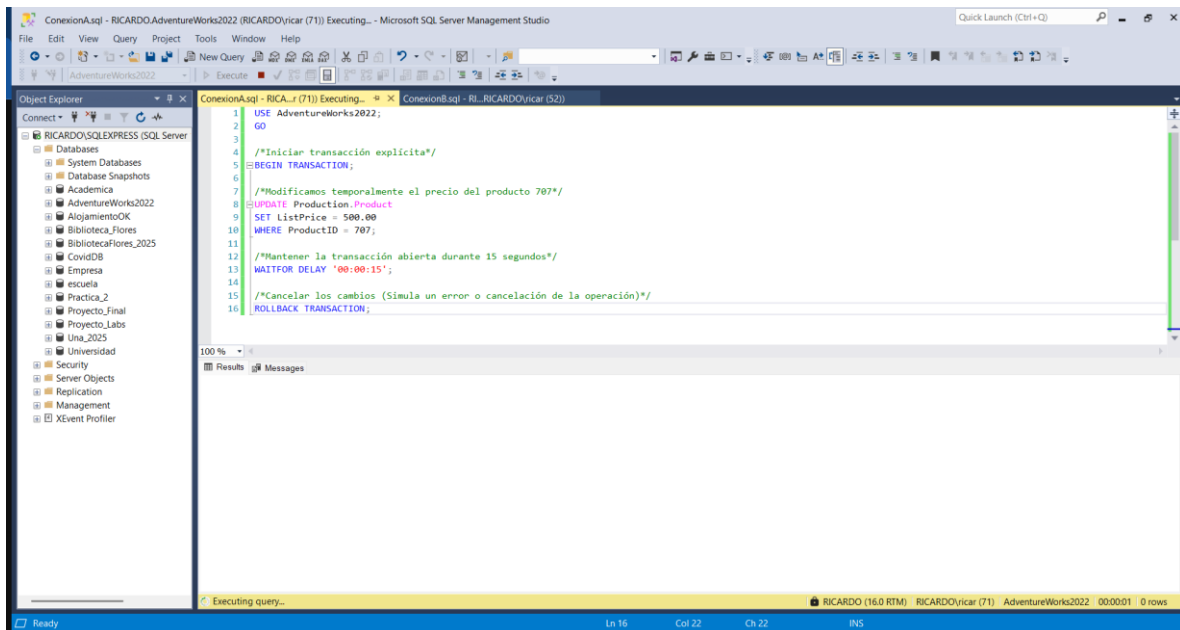


Código de Ejemplo para Production.Product

Se seleccionó la tabla **Production.Product** de la base de datos *AdventureWorks* debido a que representa un escenario crítico y común en entornos comerciales y relacionales. Al almacenar información clave de negocio como el precio de lista (ListPrice) o costos, permite simular con claridad un entorno de alta concurrencia donde una transacción actualiza temporalmente el valor de un producto mientras otro proceso intenta leer dicho dato para realizar cálculos de cotización o venta. Esto facilita demostrar de manera evidente el impacto de leer información no confirmada en un proceso transaccional.

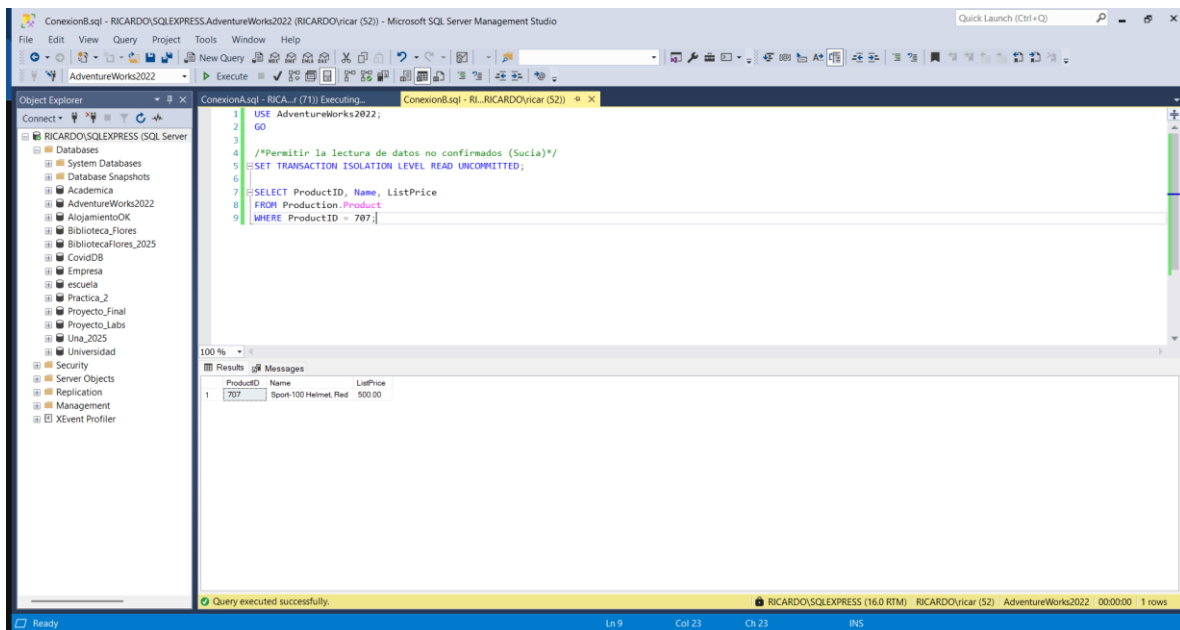
Para demostrar la falla y su comportamiento en tiempo real, se requiere simular dos conexiones independientes en SQL Server Management Studio (SSMS):

Conexión A

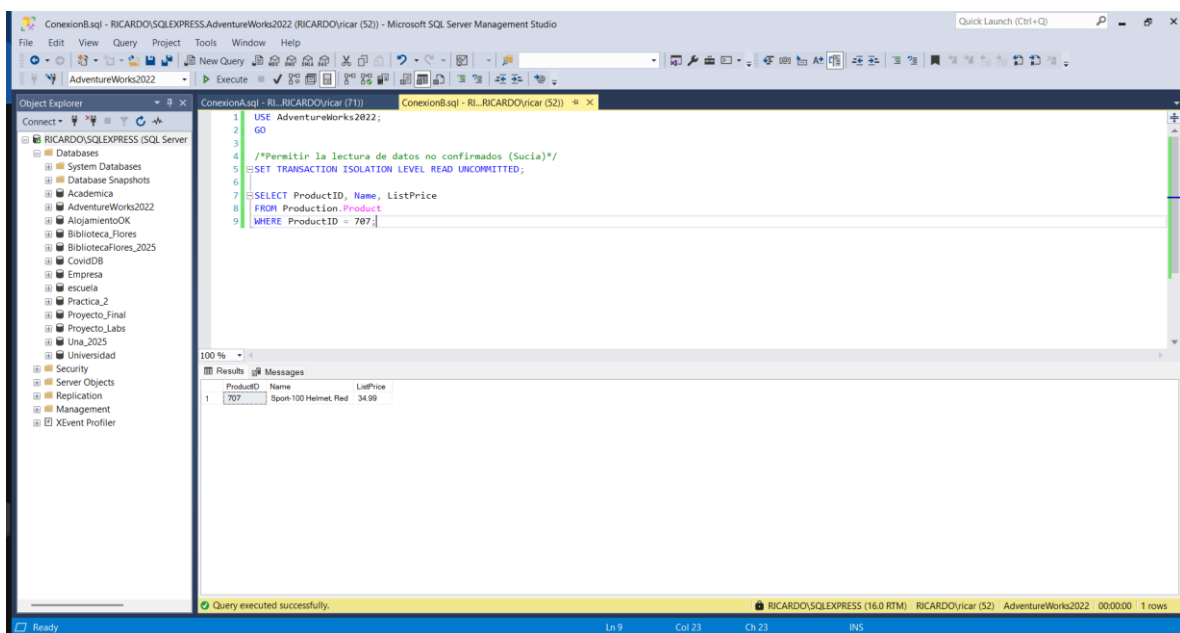


“La Conexión A es la encargada de alterar datos temporalmente y luego cancelarlos (ROLLBACK).”

Conexión B



"Lectura de un dato no confirmado (ListPrice = 500.00) bajo el nivel READ UNCOMMITTED mientras la Conexión A mantiene la transacción activa".



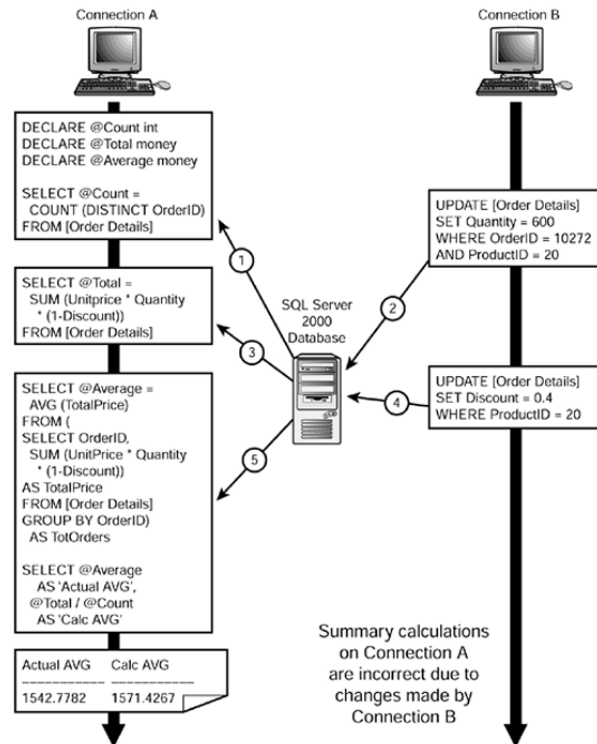
"Comprobación de inconsistencia: el precio vuelve a su valor original tras el ROLLBACK de la Conexión A, confirmando que la lectura previa fue un dato sucio/inválido".

Conclusión

En esta prueba se observó que el uso del nivel de aislamiento READ UNCOMMITTED evita la espera por bloqueos exclusivos, pero rompe el principio de Aislamiento (ACID). Esto permite que se lean cambios provisionales que pueden ser descartados por un ROLLBACK. Por otro lado, la implementación del aislamiento READ COMMITTED (por defecto en SQL Server) resuelve esta vulnerabilidad haciendo uso de bloqueos que detienen las lecturas concurrentes sobre transacciones abiertas, garantizando que el sistema opere únicamente sobre datos confirmados

Lectura no repetible (Non repeatable read).

Se produce cuando una transacción vuelve a leer los mismos datos que ya había leído previamente y encuentra que han sido modificados o eliminados por otra conexión en medio de su ejecución, obteniendo valores distintos en la misma transacción.



Conexión A:

The screenshot shows the Microsoft SQL Server Management Studio interface. The 'Object Explorer' on the left displays the database structure of 'AdventureWorks2022'. The 'Query Editor' in the center contains the following SQL script:

```
-- Primera lectura: Obtenemos el precio original del producto
SELECT ListPrice AS PrecioInicial
FROM Production.Product
WHERE ProductID = 1;

-- Pausa la ejecución por 10 segundos para darte tiempo de ejecutar la Conexión B
WAITFOR DELAY '00:00:10';

-- Segunda lectura: Volvemos a consultar dentro de la MISMA transacción
SELECT ListPrice AS PrecioFinal
FROM Production.Product
WHERE ProductID = 1;

COMMIT TRANSACTION;
```

The 'Results' pane at the bottom shows the output of the query, which was executed successfully. It displays two rows of data:

PrecioInicial
1 0.00

PrecioFinal
1 10.00

The status bar at the bottom indicates 'Query executed successfully.' and '2 rows'.

Resultado de ejecución

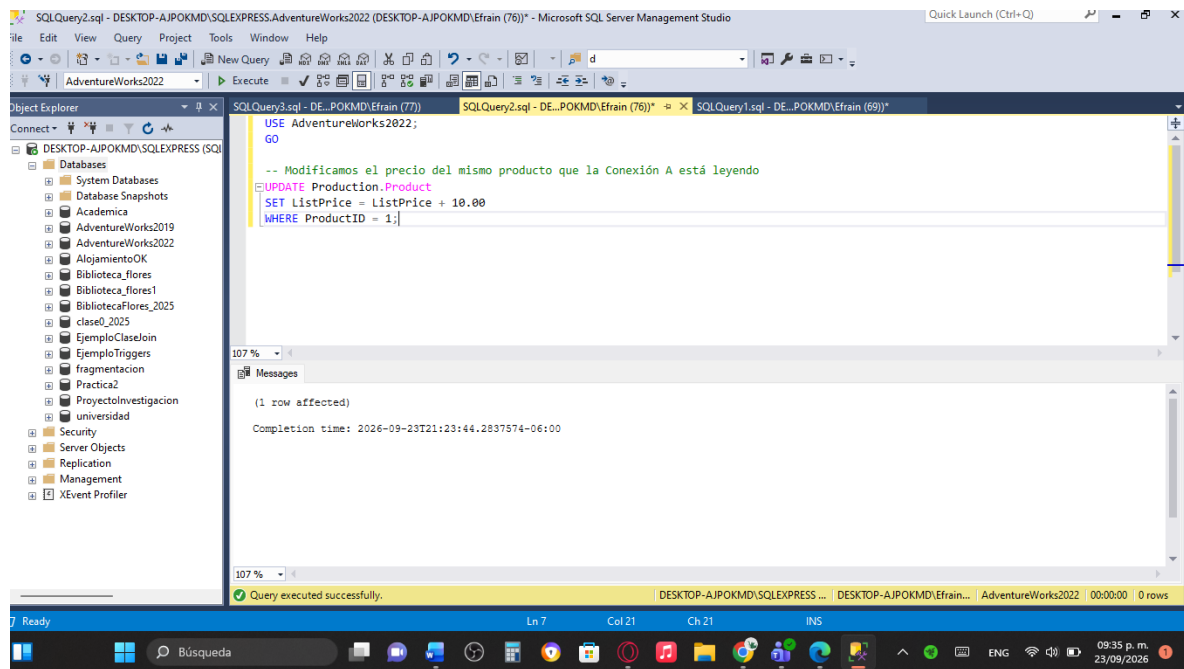
This screenshot shows the same SQL script as the previous one, but with the results of the execution displayed in the 'Results' pane. The query was executed successfully, and the results are as follows:

PrecioInicial
1 0.00

PrecioFinal
1 10.00

The status bar at the bottom confirms 'Query executed successfully.' and '2 rows'.

Conexión B:

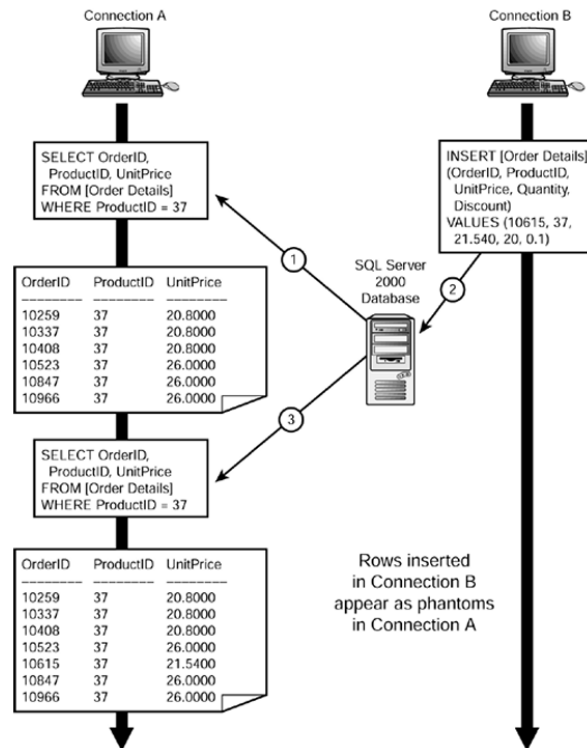


Conclusión

En este escenario se pudo observar cómo ocurre el problema de la lectura no repetible, ya que dentro de una misma transacción se ejecutaron dos consultas idénticas sobre el mismo registro, con un atraso de la segunda. En esa ventana de tiempo, otra conexión actualizó el valor del campo que afectaría la ejecución de la segunda consulta, devolviendo un precio distinto al que se leyó inicialmente. Esto ejemplifica un problema muy común e importante sobre la consistencia de los datos en transacciones concurrentes.

Lectura fantasma (Phantom Reads).

Ocurre cuando una conexión realiza una consulta por un rango de datos dentro de una transacción y, antes de finalizar, otra conexión inserta nuevas filas que cumplen con los criterios de esa consulta. Al reejecutar la lectura, aparecen registros adicionales que no existían previamente (filas "fantasma")



Conexión A:

```
USE AdventureWorks2022;
GO
-- Establecemos el nivel de aislamiento a READ COMMITTED (el valor por defecto)
--Para evitar lecturas sucias y hacer uso de bloqueos compartidos
SET TRANSACTION ISOLATION LEVEL READ COMMITTED;

BEGIN TRANSACTION;

-- Primera lectura: Contamos cuántos productos pertenecen a la Subcategoría 1
SELECT COUNT(*) AS CantidadInicial
FROM Production.Product
WHERE ProductSubcategoryID = 1;

-- Pausa la ejecución por 10 segundos para darte tiempo de ejecutar la Conexión B
WAITFOR DELAY '00:00:10';

-- Segunda lectura: Volvemos a consultar dentro de la MISMA transacción
SELECT COUNT(*) AS CantidadFinal
FROM Production.Product
WHERE ProductSubcategoryID = 1;

COMMIT TRANSACTION
```

Resultado de ejecución

	CantidadInicial
1	32

	CantidadFinal
1	33

Conexión B:

```
-- Insertamos un registro nuevo que cumple con el criterio de la consulta de la Conexión A
INSERT INTO Production.Product (
    Name,
    ProductNumber, MakeFlag, FinishedGoodsFlag, Color, SafetyStockLevel, ReorderPoint,
    StandardCost, ListPrice, DaysToManufacture, ProductSubcategoryID, SellStartDate
)
VALUES (
    'Producto Fantasma Prueba', 'FANT-001', 0, 0, 'Black', 1000, 750,
    0.00, 0.00, 0, 1, GETDATE()
);
```

Conclusión

En este escenario se pudo observar como ocurre el problema de la lectura fantasma, ya que dentro de una misma transacción se ejecutaron dos consultas iguales, con un atraso de la segunda, en esa ventana de tiempo se insertaron campos que afectarían la ejecución de la segunda consulta, apareciendo filas nuevas que la transacción de la inserción realizó, esto ejemplifica un problema muy común e importante sobre las consultas realizadas a bases de datos.